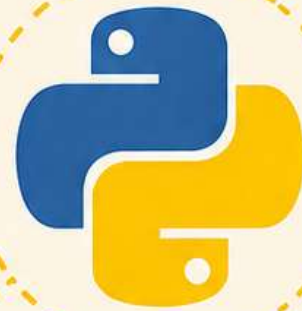


04  
PHASE

# Python for Data Analyst

Code. Analyze. Automate.  
Build **Real Impact**.



Data Handling  
with Pandas



Data Analysis  
& Visualization



Automation  
& Workflows



Build Scalable  
Solutions



Transform Data. Write Python. Drive Decisions.

## Unit 1: Python Fundamentals

---

### Topics Covered

- 1.1 Variables, Constants, and PEP 8 Naming Conventions
- 1.2 Primitive Data Types: String, Integer, Float, Boolean
- 1.3 String Manipulation: f-strings, .strip(), .split(), .replace()
- 1.4 Type Casting and Checking (type(), isinstance())
- 1.5 Mathematical Operators & Order of Operations
- 1.6 Logical Operators (and, or, not) and Membership (in)
- 1.7 Conditional Logic: if, elif, else
- 1.8 Comparison Operators for Data Filtering



LEARN SMART. ANALYZE BETTER. GROW FASTER.

# Unit 1: Python Fundamentals for Analytics

## 1.1 Variables, Constants, and PEP 8 Naming Conventions

### Definition

A variable is a named container that stores a value in memory. A constant is a variable whose value is not meant to change. PEP 8 is Python's official style guide that defines how to write clean, readable code.

### Sub-Topics

- Variable: A label attached to a value (e.g., `age = 25`)
- Constant: By convention, written in ALL\_CAPS (e.g., `MAX_ROWS = 1000`)
- PEP 8 Naming: snake\_case for variables/functions, UPPER\_SNAKE\_CASE for constants, PascalCase for classes
- Variable Reassignment: A variable can point to a new value at any time
- Multiple Assignment: Assign multiple variables in one line (e.g., `x, y = 10, 20`)

### Why Important for Data Analysts

Data analysts constantly create and name variables to store data — column names, counts, thresholds, etc. Consistent naming (PEP 8) makes your code understandable to teammates and your future self.

### Real-World Analogy

Think of a variable like a labelled sticky note on a whiteboard. You write a label (e.g., 'Total Sales') and stick a number on it. You can peel it off and replace the number anytime. A constant is a sticky note on a laminated board — you are not supposed to change it.

### Syntax

```
# Variable assignment
variable_name = value

# Constant (convention only - Python doesn't enforce it)
MAX_RECORDS = 10000

# Multiple assignment
city, country = 'Mumbai', 'India'
```

### Example

```
# Analyst tracking quarterly sales
total_sales = 450000 # variable
TAX_RATE = 0.18 # constant
region, quarter = 'West', 'Q3'

print(total_sales) # 450000
print(TAX_RATE) # 0.18
print(region) # West
```

## Industry Use Case

In a retail analytics project, a data analyst creates variables like `monthly_revenue``, `discount_rate``, and `FISCAL_YEAR_START`` to store and reuse key business metrics throughout a script. PEP 8 naming ensures the script is reviewed and maintained easily by the whole analytics team.

## 1.2 Primitive Data Types: String, Integer, Float, Boolean

### Definition

Python has four basic (primitive) data types: String (text), Integer (whole numbers), Float (decimal numbers), and Boolean (True/False). Every piece of data you work with belongs to one of these types.

### Sub-Topics

- String (str): Sequence of characters enclosed in quotes — e.g., `'Delhi'`, `"Sales"`
- Integer (int): Whole numbers without decimals — e.g., `100`, `-5`, `0`
- Float (float): Numbers with decimal points — e.g., `3.14`, `99.99`, `-0.5`
- Boolean (bool): Only two possible values — `True` or `False`; used in conditions
- NoneType: A special type with only one value — `None` (represents 'no value')

### Why Important for Data Analysts

Every dataset column has a data type. If you try to add a string to a number, Python throws an error. Knowing data types helps you clean data, avoid bugs, and choose the right operations.

### Real-World Analogy

Imagine a filing cabinet. Each drawer is labelled differently: one holds text documents (str), one holds whole numbers like employee IDs (int), one holds decimal values like salaries (float), and one holds yes/no flags (bool). You must put the right paper in the right drawer.

### Syntax

```
# String
name = 'Priya'

# Integer
age = 30

# Float
salary = 75000.50

# Boolean
is_active = True
```

### Example

```
customer_name = 'Rohan Mehta' # str
order_id = 10245 # int
order_value = 1399.99 # float
```



```
is_premium = True # bool

print(type(customer_name)) # <class 'str'>
print(type(order_id)) # <class 'int'>
print(type(order_value)) # <class 'float'>
print(type(is_premium)) # <class 'bool'>
```

### Industry Use Case

In an e-commerce dataset, `product\_name` is a string, `quantity\_sold` is an integer, `unit\_price` is a float, and `in\_stock` is a boolean. Recognising types early allows analysts to apply correct aggregations — summing floats, counting integers, and filtering on booleans.

## 1.3 String Manipulation: f-strings, .strip(), .split(), .replace()

### Definition

String manipulation means modifying, formatting, or extracting information from text data. Python provides many built-in string methods and f-strings (formatted string literals) for this purpose.

### Sub-Topics

- f-strings: Embed variables/expressions inside strings using `f'text {variable}'`
- `.strip()`: Remove leading and trailing whitespace (or specific characters)
- `.split()`: Split a string into a list based on a separator (default: space)
- `.replace()`: Replace all occurrences of a substring with another
- Other useful methods: `.upper()`, `.lower()`, `.title()`, `.startswith()`, `.endswith()`

### Why Important for Data Analysts

Real-world data is messy — names have extra spaces, phone numbers include dashes, addresses are inconsistently formatted. String manipulation is one of the most frequently used skills in data cleaning.

### Real-World Analogy

Think of a string like a sentence written on a whiteboard. `.strip()` is an eraser that cleans the edges. `.split()` is scissors that cut the sentence into words. `.replace()` is correction fluid — white out one word and write another. An f-string is a template letter with blank spaces that you fill in.

### Syntax

```
# f-string
f'Hello, {variable_name}!'

# strip - remove whitespace
text.strip()

# split - returns a list
text.split(',')
```

```
# replace
text.replace('old', 'new')
```

### Example

```
name = ' Sneha Patil '
email = 'sneha.patil@example.com'
tags = 'python,pandas,sql'

# f-string
greeting = f'Welcome, {name.strip()}!'
print(greeting) # Welcome, Sneha Patil!

# strip
print(name.strip()) # 'Sneha Patil'

# split
print(tags.split(',')) # ['python', 'pandas', 'sql']

# replace
clean_email = email.replace('@example.com', '')
print(clean_email) # sneha.patil
```

### Industry Use Case

A healthcare analyst receives patient data where names are padded with spaces and diagnoses are stored as comma-separated strings. Using `.strip()` on names, `.split(',')` on diagnosis lists, and f-strings to generate patient summary reports makes the pipeline clean and reusable.

## 1.4 Type Casting and Checking (`type()`, `isinstance()`)

### Definition

Type casting (or type conversion) means converting a value from one data type to another. Type checking means verifying what type a value currently is.

### Sub-Topics

- `type()`: Returns the data type of any variable
- `isinstance()`: Checks if a variable belongs to a specific type — returns True/False
- Implicit Casting: Python automatically converts types in some operations (`int + float = float`)
- Explicit Casting: You manually convert — `int()`, `float()`, `str()`, `bool()`
- Common conversions: `int('25') → 25`, `str(100) → '100'`, `float('3.14') → 3.14`

### Why Important for Data Analysts

Data imported from CSV files is often read as strings — even numbers. Before doing calculations, you must convert them to int or float. Type checking prevents errors before they happen.

## Real-World Analogy

Imagine you receive a parcel with coins inside but wrapped in a box labelled 'Papers'. You must open it, see what's really inside, and re-label it correctly. ``type()``` is reading the label; type casting is re-labelling the box so it can be used correctly.

## Syntax

```
# Check type
type(variable)

# Check with isinstance
isinstance(variable, int) # True or False

# Cast to integer
int('42')

# Cast to float
float('3.14')

# Cast to string
str(100)
```

## Example

```
# Simulating CSV-imported data (everything is a string)
age_str = '28'
price_str = '499.99'
in_stock = 'True'

print(type(age_str)) # <class 'str'>
print(isinstance(age_str, str)) # True

# Convert for calculations
age = int(age_str) # 28
price = float(price_str) # 499.99
stock = bool(in_stock) # True

print(age + 2) # 30
print(price * 1.18) # 589.9882
```

## Industry Use Case

A finance analyst loads a CSV of transaction records. All columns — including ``amount``, ``quantity``, and ``date_year`` — are imported as strings. Using ``float()``` on amount and ``int()``` on year and quantity allows correct aggregation and date-filtering logic.

## 1.5 Mathematical Operators & Order of Operations

### Definition

Python supports standard mathematical operators for arithmetic calculations. The order of operations (BODMAS/PEMDAS) determines which operations are evaluated first.

## Sub-Topics

- `+` Addition, `-` Subtraction, `*` Multiplication, `/` Division (returns float)
- `//` Floor Division: divides and rounds down to the nearest integer
- `%` Modulo: returns the remainder after division
- `**` Exponentiation: raises a number to a power
- Order of Operations: `**` first, then `*` `/` `//` `%`, then `+` `-`; use `()` to override
- Augmented Assignment: `x += 5` is shorthand for `x = x + 5`

## Why Important for Data Analysts

Data analysts calculate KPIs, averages, growth rates, percentages, and statistical formulas daily. Understanding operator precedence prevents calculation bugs that can mislead business decisions.

## Real-World Analogy

Python follows the same BODMAS rule you learned in school. Think of parentheses as a VIP queue — whatever is inside gets served first. Multiplication is like a pushy customer who always goes before addition.

## Syntax

```
a + b # Addition
a - b # Subtraction
a * b # Multiplication
a / b # Division (float result)
a // b # Floor division (integer result)
a % b # Modulo (remainder)
a ** b # Exponentiation
```

## Example

```
revenue = 500000
expenses = 320000
months = 12

profit = revenue - expenses # 180000
monthly_avg = revenue / months # 41666.666...
monthly_floor = revenue // months # 41666
remainder = revenue % months # 8 (leftover)
growth = 1.05 ** 3 # 5% growth over 3 years = 1.157

# Order of operations
result = 2 + 3 * 4 # 14 (not 20!)
result = (2 + 3) * 4 # 20 (parentheses first)
```

## Industry Use Case

A supply chain analyst calculates stock replenishment: ``units_needed = total_demand - current_stock``. They use ``//`` to find full cartons needed, ``%`` to find loose units, and ``**`` to model exponential demand growth. Correct operator order ensures the right quantity is ordered.



## 1.6 Logical Operators (and, or, not) and Membership (in)

### Definition

Logical operators combine multiple conditions and return True or False. The ``in`` operator checks whether a value exists within a sequence (list, string, etc.).

### Sub-Topics

- `and`: Returns True only if BOTH conditions are True
- `or`: Returns True if AT LEAST ONE condition is True
- `not`: Reverses the Boolean — `not True = False`
- `in`: Returns True if a value is found in a list, string, or other iterable
- `not in`: Returns True if a value is NOT found in the sequence
- Short-circuit evaluation: Python stops checking as soon as the result is determined

### Why Important for Data Analysts

Filtering is the heart of data analysis. You constantly ask 'Show me records where region is East AND sales > 50000.' Logical operators let you write complex filters in a single readable condition.

### Real-World Analogy

``and`` is like a security door that needs both a keycard AND a fingerprint. ``or`` is like a multi-door building — enter through any one door. ``not`` is an inverter switch — it flips the signal. ``in`` is like checking a guest list — 'Is this name on the list?'

### Syntax

```
# and — both must be True
condition1 and condition2

# or — at least one must be True
condition1 or condition2

# not — reverses True/False
not condition

# in — membership check
value in [list_or_string]
```

### Example

```
region = 'South'
sales = 75000
status = 'Pending'
valid_regions = ['North', 'South', 'East', 'West']

# and
print(region == 'South' and sales > 50000) # True

# or
```

```
print(status == 'Approved' or sales > 60000) # True

# not
print(not status == 'Approved') # True

# in
print(region in valid_regions) # True
print('North' not in valid_regions) # False
```

### Industry Use Case

An HR analyst filters employee records to find: employees who are `active AND in the 'Sales' department AND salary > 50000`. Using `and` with `in` lets them write a single readable filter condition instead of multiple nested checks.

## 1.7 Conditional Logic: if, elif, else

### Definition

Conditional logic allows your program to make decisions. Python evaluates a condition and runs different blocks of code depending on whether it is True or False.

### Sub-Topics

- **if**: Runs a block of code only when the condition is True
- **elif** (else if): Checks another condition if the previous **if** was False
- **else**: Runs when all above conditions are False — acts as a fallback
- Nested conditions: An **if** block can contain another **if** inside
- Ternary expression: `value_if_true if condition else value_if_false` — one-liner shorthand

### Why Important for Data Analysts

Data analysis involves classification, labelling, and routing logic. For example, marking orders as 'High Value' or 'Low Value', assigning risk scores, or categorising customer segments all rely on conditional logic.

### Real-World Analogy

Think of a traffic signal. `if` the signal is green, go. `elif` it is yellow, slow down. `else` (it must be red), stop. Python works the same way — it evaluates each condition in order and does exactly what the matching branch says.

### Syntax

```
if condition:
    # runs if condition is True
elif another_condition:
    # runs if first was False and this is True
else:
    # runs if ALL above were False
```

### Example

```

order_value = 8500

if order_value >= 10000:
    category = 'Platinum'
elif order_value >= 5000:
    category = 'Gold'
elif order_value >= 1000:
    category = 'Silver'
else:
    category = 'Standard'

print(f'Order Category: {category}') # Order Category: Gold

# Ternary (one-liner)
label = 'High' if order_value > 7000 else 'Low'
print(label) # High

```

### Industry Use Case

A banking analyst segments loan applicants: if `credit\_score >= 750`, mark 'Low Risk'; elif `>= 600`, mark 'Medium Risk'; else 'High Risk'. This conditional labelling feeds into risk dashboards and automated approval workflows.

## 1.8 Comparison Operators for Data Filtering

### Definition

Comparison operators compare two values and return True or False. They are the foundation of all data filtering — every filter condition you write uses at least one comparison operator.

### Sub-Topics

- `==` Equal to: checks if two values are the same
- `!=` Not equal to: checks if two values are different
- `>` Greater than, `<` Less than
- `>=` Greater than or equal to, `<=` Less than or equal to
- Chained comparisons: `10 < age < 60` — Python allows elegant range checks
- Comparing strings: Python compares strings alphabetically (lexicographically)

### Why Important for Data Analysts

Every filter in pandas, SQL, or plain Python uses comparison operators. `df[df['salary'] > 50000]` relies on `>`. Understanding these operators precisely prevents off-by-one errors in thresholds that could misclassify thousands of records.

### Real-World Analogy

Comparison operators are like a weighing scale. `==` checks if both sides balance perfectly. `>` and `<` tell you which side is heavier. `!=` is an alarm that rings when the two sides do NOT match. You use these scales thousands of times in analysis to decide which data makes the cut.

## Syntax

```
a == b # Equal
a != b # Not equal
a > b # Greater than
a < b # Less than
a >= b # Greater than or equal to
a <= b # Less than or equal to

# Chained comparison
low <= value <= high
```

## Example

```
age = 35
salary = 55000
dept = 'Finance'

print(age == 35) # True
print(dept != 'HR') # True
print(salary > 50000) # True
print(salary <= 60000) # True

# Chained comparison
print(30 <= age <= 40) # True — elegant range check

# Used in filtering (plain Python)
employees = [
    {'name': 'Asha', 'dept': 'Finance', 'salary': 62000},
    {'name': 'Ravi', 'dept': 'HR', 'salary': 45000},
    {'name': 'Meena', 'dept': 'Finance', 'salary': 71000},
]
finance_high = [e for e in employees
    if e['dept'] == 'Finance' and e['salary'] >= 60000]
print(finance_high)
# [{'name': 'Asha', ...}, {'name': 'Meena', ...}]
```

## Industry Use Case

A logistics analyst needs to find all shipments where `delivery\_days > 7` (delayed) AND `region == 'North'`. Comparison operators power this query. In pandas, `df[(df['delivery\_days'] > 7) & (df['region'] == 'North')]` uses the same logic to filter thousands of rows instantly.





## Loved This Chapter? **There's More.**

Explore our complete range of Structured Handbook — each course is crisp, exam-ready, and built for working professionals.

★ All Courses Starting at ₹399\* onwards ★

### — OUR STRUCTURED HANDBOOK COURSES —



#### Data Analytics

All 8 Phases



#### Data Science

Zero → Pro



#### Python for Data

Practical Handbook



#### Power BI & Tableau

Visual Insights



#### Excel for Analysts

Shortcuts + Formulas



#### Statistics & Probability

Concept + MCQs



#### Machine Learning

No Maths Overload



#### Interview Prep Bundle

2000+ Qs

Get All Courses & Handbook at



<https://learnlyticshandbook.vercel.app/>

*Learn Smart. Analyze Better. Grow Faster.*

## Unit 1 — Test Questions

Topics: Variables · Data Types · Strings · Type Casting · Operators · Logic · Conditionals · Comparison

### Level 1: Warmup Test — 5 MCQ Questions

Answer all 5 questions. Each has one correct answer. The correct answer is highlighted in green.

**Q1** According to PEP 8, which of the following is the correct way to name a constant in Python?

- A) totalRevenue
- B) total\_revenue
- C) TOTAL\_REVENUE
- D) TotalRevenue

**Q2** What is the data type of the value returned by the expression: `type(3.14)`?

- A) float
- B) int
- C) `<class 'float'>`
- D) str

**Q3** What will the following code output? `tags = 'python,pandas,sql' print(tags.split(','))`

- A) 'python pandas sql'
- B) ['python', 'pandas', 'sql']
- C) ('python', 'pandas', 'sql')
- D) {'python', 'pandas', 'sql'}

**Q4** What does the expression `2 + 3 * 4` evaluate to in Python?

- A) 20
- B) 14
- C) 24
- D) 10

**Q5** Which operator should be used to check if the variable region exists inside the list `valid_regions`?

- A) `==`
- B) `contains`
- C) `in`
- D) `has`

## Level 2: Interviewer Test — 15 Intermediate / Semi-Advanced Questions

*This section tests conceptual depth, practical thinking, and coding skill. Answer using a conversational, student-style approach — explain the "why", give examples, and keep it practical.*

**Q1:** What is the difference between a variable and a constant in Python? How does PEP 8 help teams write more maintainable data analytics code?

**Q2:** Explain the four primitive data types in Python with examples relevant to a business dataset. Why is knowing the difference between `int` and `float` critical before performing calculations?

**Q3:** What is type casting in Python? Explain implicit vs explicit casting with an example from a CSV data loading scenario.

**Q4:** Explain how conditional logic (if/elif/else) and comparison operators work together in data classification tasks. Include an example of a ternary expression.

LEARN SMART. ANALYZE BETTER. GROW FASTER.

 Section B: Scenario-Based Descriptive Questions (4 Questions)

**Q1:** Scenario: You are an HR analyst at a large company. You receive a CSV file of employee records. The 'salary' column is loaded as strings, and the 'department' column has leading/trailing spaces. How would you clean and prepare this data using Python basics from Unit 1?

**Q2:** Scenario: You are building a risk classification engine for a bank's loan department. Write the classification logic for loan applicants using if/elif/else based on credit score, and explain why the order of conditions matters.

**Q3:** Scenario: A supply chain analyst needs to flag shipments that are either delayed (delivery\_days > 7) OR high-value (shipment\_value > 100000) AND destined for the North region. Write the filtering logic and explain how logical operators affect the result.



**Q4:** Scenario: You are a retail analyst and receive a product dataset where prices are stored as strings with a currency symbol (e.g., '₹1,399.99'). How would you use string manipulation and type casting to extract clean numeric prices for calculation?

### Section C: Coding Questions (4 Questions)

**Q1:** Write a Python function `calculate_profit_margin(revenue, expenses)` that calculates the profit margin as a percentage. It should handle zero revenue gracefully and return the result rounded to 2 decimal places.

LEARN SMART. ANALYZE BETTER. GROW FASTER.

**Q2:** Write a Python function `classify_employee(salary, department, is_active)` that uses `if/elif/else` and logical operators to return a classification string: 'Senior Active' (salary  $\geq$  80000 AND active), 'Junior Active' (salary  $<$  80000 AND active), or 'Inactive' (not active).

**Q3:** Write a Python script that takes a raw list of sales records (each a dict with 'region', 'sales', 'month') and uses string manipulation, comparison operators, and logical operators to filter records where sales > 50000 AND region is either 'North' or 'South', then prints an f-string summary for each.

**Q4:** Write a function `parse_employee_record(record)` that takes a dict with string values (from a CSV import), validates types using `isinstance()`, casts each field, and returns a clean dict. Fields: 'emp\_id' (int), 'name' (str, stripped), 'salary' (float), 'is\_active' (bool).

#### Section D: Database Scenario-Based Coding (3 Questions)

*These questions simulate working with database-like structured data (list of dicts) using Python fundamentals — the precursor to pandas and SQL queries.*

**Q1:** Database Scenario: You have a transactions table (list of dicts) with fields: transaction\_id (int), customer\_name (str), amount (float), status (str), region (str). Write Python code to: (1) filter all 'Completed' transactions from the 'West' region with amount > 10000, and (2) calculate the total amount for those records.

**Q2:** Database Scenario: You have an employees table. Write Python code to group employees by department and calculate the average salary per department. The result should be a dict mapping department name to average salary (rounded to 2 decimal places).

**Q3:** Database Scenario: You have an orders table and a customers table. Simulate a JOIN in Python: for each order, find the matching customer name, then produce a combined record. Filter the result to show only orders where the customer is 'Premium' tier AND order\_value > 5000.

## Unit 1 — Test Questions Answers

Topics: Variables · Data Types · Strings · Type Casting · Operators · Logic · Conditionals · Comparison

### 🌀 Level 1: Warmup Test — 5 MCQ Questions

Answer all 5 questions. Each has one correct answer. The correct answer is highlighted in green.

**Q1** According to PEP 8, which of the following is the correct way to name a constant in Python?

- A) totalRevenue
- B) total\_revenue
- C) TOTAL\_REVENUE ✓ Correct**
- D) TotalRevenue

**Q2** What is the data type of the value returned by the expression: `type(3.14)`?

- A) float
- B) int
- C) `<class 'float'>` ✓ Correct**
- D) str

**Q3** What will the following code output? `tags = 'python,pandas,sql' print(tags.split(','))`

- A) 'python pandas sql'
- B) ['python', 'pandas', 'sql'] ✓ Correct**
- C) ('python', 'pandas', 'sql')
- D) {'python', 'pandas', 'sql'}

**Q4** What does the expression `2 + 3 * 4` evaluate to in Python?

- A) 20
- B) 14 ✓ Correct**
- C) 24
- D) 10

**Q5** Which operator should be used to check if the variable region exists inside the list `valid_regions`?

- A) `==`
- B) `contains`
- C) `in` ✓ Correct**
- D) `has`



# Learnlytics

---

.handbook

LEARN SMART. ANALYZE BETTER. GROW FASTER.



## Level 2: Interviewer Test — 15 Intermediate / Semi-Advanced Questions

*This section tests conceptual depth, practical thinking, and coding skill. Answer using a conversational, student-style approach — explain the "why", give examples, and keep it practical.*

**Q1:** What is the difference between a variable and a constant in Python? How does PEP 8 help teams write more maintainable data analytics code?

**Answer:** So, I think of a variable as a labelled box you can open and swap the contents of at any time. For example, when tracking monthly revenue, I might write `monthly_revenue = 450000` — and next month I simply reassign it. A constant, on the other hand, is a box you agree not to open — like `TAX_RATE = 0.18`. Python doesn't actually enforce this; it's purely a team convention backed by PEP 8's `UPPER_SNAKE_CASE` rule.

**Answer:** Why does PEP 8 matter in analytics teams? Because scripts get handed off. If one analyst writes `mr` and another writes `MonthlyRev`, the next person wastes time decoding intent. PEP 8 aligns everyone on `snake_case` for variables, `UPPER_SNAKE_CASE` for constants, and `PascalCase` for classes — making code feel like a shared language, not a personal diary.

```
# Clean PEP 8 example
monthly_revenue = 450000 # variable - changes each month
TAX_RATE = 0.18 # constant - agreed not to change
region, quarter = 'West', 'Q3' # multiple assignment
```

**Q2:** Explain the four primitive data types in Python with examples relevant to a business dataset. Why is knowing the difference between `int` and `float` critical before performing calculations?

**Answer:** Great question — data types are the foundation of clean analysis. In a real e-commerce dataset, every column has a natural type: `product_name` is a `str`, `quantity_sold` is an `int`, `unit_price` is a `float`, and `in_stock` is a `bool`.

**Answer:** The `int` vs `float` distinction matters a lot. If I write `total = quantity * price` and `quantity` is accidentally a string (imported from CSV), Python throws a `TypeError` immediately. More subtly, dividing two integers in Python 3 gives a `float` — so `7 / 2 = 3.5`, not `3`. But `7 // 2 = 3`. Getting this wrong in a stock calculation could mean ordering 3 cartons when you need 4.



**Answer:** I also always keep `NoneType` in mind — it appears when a field is missing. Treating `None` as `0` silently corrupts aggregations, so I check `isinstance(value, type(None))` before using it.

```
customer_name = 'Rohan Mehta' # str
order_id = 10245 # int
order_value = 1399.99 # float
is_premium = True # bool
missing_field = None # NoneType - handle explicitly
```

**Q3:** What is type casting in Python? Explain implicit vs explicit casting with an example from a CSV data loading scenario.

**Answer:** Type casting is converting a value from one type to another. Python does implicit casting automatically in safe situations — for instance, adding an `int` and a `float` gives a `float` without any instruction from me. But explicit casting is where I actively intervene using `int()`, `float()`, `str()`, or `bool()`.

**Answer:** The most practical scenario is CSV loading. When pandas or even Python's `csv` module reads a file, EVERY value comes in as a string — even `'28'` or `'499.99'`. I can't do arithmetic on strings. I need to explicitly convert them.

**Answer:** Why use `isinstance()` first? Because if a field is empty or `'N/A'`, calling `float('N/A')` raises a `ValueError`. So I defensively check `isinstance(value, str)` and handle edge cases before casting.

```
# Simulating CSV-imported data
age_str = '28'
price_str = '499.99'

print(isinstance(age_str, str)) # True - confirm it's a string

age = int(age_str) # 28 - explicit cast
price = float(price_str) # 499.99

print(age + 2) # 30 - now arithmetic works
print(price * 1.18) # 589.9882 - correct GST calculation
```

**Q4:** Explain how conditional logic (`if/elif/else`) and comparison operators work together in data classification tasks. Include an example of a ternary expression.

**Answer:** Conditional logic is the decision-making engine of any analytics pipeline. I use it to classify, label, and route data. Comparison operators (`==`, `!=`, `>`, `<`, `>=`, `<=`) are the questions I ask — conditionals are how I respond to the answers.

**Answer:** A classic use case is customer tier segmentation. For every order, I ask: is the value `>= 10000`? If yes, Platinum. If not, is it `>= 5000`? Gold. And so on. The if/elif/else chain maps cleanly to that business rule.

**Answer:** What I appreciate about Python is chained comparisons — I can write `1000 <= order_value < 5000` as a single readable check instead of two separate conditions joined with `and`.

**Answer:** For quick binary labels, a ternary expression is elegant and keeps my code compact without losing readability:

```
order_value = 8500
```

```
if order_value >= 10000:
    category = 'Platinum'
elif order_value >= 5000:
    category = 'Gold'
elif order_value >= 1000:
    category = 'Silver'
else:
    category = 'Standard'
```

```
# Ternary - one-liner for binary labels
label = 'High' if order_value > 7000 else 'Low'
print(label) # High
```

handbook

## Section B: Scenario-Based Descriptive Questions (4 Questions)

**Q1:** Scenario: You are an HR analyst at a large company. You receive a CSV file of employee records. The 'salary' column is loaded as strings, and the 'department' column has leading/trailing spaces. How would you clean and prepare this data using Python basics from Unit 1?

**Answer:** This is a very realistic cleaning problem. My first instinct is to tackle the two issues separately: string-to-number conversion and whitespace removal.

**Answer:** For salary, every value like '75000.50' needs float() conversion. I also need to guard against empty cells — I'd check if value.strip() before casting to avoid float("") crashing my script.

**Answer:** For department, .strip() removes the sneaky leading/trailing spaces that cause mismatches. 'Finance ' and 'Finance' look the same to the eye but are different strings to Python — so a filter like dept == 'Finance' silently drops records if I forget to strip.

**Answer:** Then I'd build a cleaned list using a comprehension with type-safe casting:

```
raw_employees = [
    {'name': 'Asha ', 'dept': 'Finance ', 'salary': '75000.50'},
    {'name': 'Ravi', 'dept': 'HR ', 'salary': '45000'},
]
```

```
cleaned = [
    {
        'name': e['name'].strip(),
        'dept': e['dept'].strip(),
        'salary': float(e['salary']) if e['salary'].strip() else 0.0
    }
    for e in raw_employees
]
print(cleaned)
# [{'name': 'Asha', 'dept': 'Finance', 'salary': 75000.5}, ...]
```

**Answer:** After cleaning I'd use isinstance() checks before any arithmetic to be absolutely safe — especially if this script runs in production.

**Q2:** Scenario: You are building a risk classification engine for a bank's loan department. Write the classification logic for loan applicants using if/elif/else based on credit score, and explain why the order of conditions matters.

**Answer:** I love this scenario — it shows exactly why conditional logic order is critical. The rule set is: credit\_score >= 750 → 'Low Risk', >= 600 → 'Medium Risk', else → 'High Risk'.

**Answer:** The key insight is that Python evaluates if/elif blocks top-down and stops at the FIRST True condition. If I accidentally put the >= 600 check before >= 750, a score of 780 would be labelled 'Medium Risk' because 780 >= 600 is True and Python never reaches the Platinum check. This is a silent logic bug — no error, wrong output.

**Answer:** So I always order conditions from most restrictive (highest threshold) to least restrictive:

```
def classify_loan_risk(credit_score):
    if credit_score >= 750:
        return 'Low Risk'
    elif credit_score >= 600:
        return 'Medium Risk'
    else:
        return 'High Risk'
```

```
# Testing
scores = [780, 640, 520]
for s in scores:
    print(f'Score {s}: {classify_loan_risk(s)}')
# Score 780: Low Risk
# Score 640: Medium Risk
# Score 520: High Risk
```

**Answer:** In production I'd also add a guard: `if not isinstance(credit_score, (int, float))` to handle `None` or string inputs from a database query — because real data always has surprises.

**Q3:** Scenario: A supply chain analyst needs to flag shipments that are either delayed (`delivery_days > 7`) OR high-value (`shipment_value > 100000`) AND destined for the North region. Write the filtering logic and explain how logical operators affect the result.

**Answer:** This is a great test of operator precedence. The natural language says 'delayed OR high-value AND North' — but in Python, AND binds tighter than OR. So `delayed or high_value and north` actually evaluates as `delayed or (high_value and north)`. I need to use parentheses to express my actual intent clearly.

**Answer:** Let me define the intent: flag if (delayed OR high-value) AND North. The parentheses around the OR are critical:

```
shipments = [
    {'id': 1, 'days': 9, 'value': 50000, 'region': 'North'},
    {'id': 2, 'days': 3, 'value': 150000, 'region': 'North'},
    {'id': 3, 'days': 10, 'value': 40000, 'region': 'South'},
    {'id': 4, 'days': 2, 'value': 80000, 'region': 'East'},
]
```

```
flagged = [
    s for s in shipments
```

```
if (s['days'] > 7 or s['value'] > 100000) and s['region'] == 'North':
    ]
```

```
for s in flagged:
    print(f"Shipment {s['id']} flagged")
    # Shipment 1 flagged (delayed AND North)
    # Shipment 2 flagged (high-value AND North)
```

**Answer:** Shipment 3 is NOT flagged — delayed, but South. Shipment 4 is NOT flagged — not delayed, not high-value, and wrong region. The parentheses are doing the heavy lifting here.

**Q4:** Scenario: You are a retail analyst and receive a product dataset where prices are stored as strings with a currency symbol (e.g., '₹1,399.99'). How would you use string manipulation and type casting to extract clean numeric prices for calculation?

**Answer:** This is a very common messy-data problem. The challenge is chained cleaning: I need to strip the ₹ symbol, remove commas (thousands separators), strip whitespace, then cast to float.

**Answer:** I'd approach it step by step, and then wrap it in a reusable function with error handling — because some cells might be blank or have unexpected formats:

```
raw_prices = ['₹1,399.99', '₹ 24,999.00', '₹499', '', None]
```

```
def clean_price(raw):
    if not raw: # handles None and empty string
        return 0.0
    cleaned = raw.replace('₹', '') # remove currency symbol
    cleaned = cleaned.replace(',', '') # remove thousands separator
    cleaned = cleaned.strip() # remove whitespace
    return float(cleaned) if cleaned else 0.0
```

```
prices = [clean_price(p) for p in raw_prices]
print(prices) # [1399.99, 24999.0, 499.0, 0.0, 0.0]
```

```
# Now I can calculate
total = sum(prices)
avg = total / len([p for p in prices if p > 0])
print(f'Total: ₹{total:,.2f}') # Total: ₹26,897.99
print(f'Average: ₹{avg:,.2f}') # Average: ₹8,965.99
```

**Answer:** The f-string with `:.2f` format spec is a bonus — it formats the output back with commas and 2 decimal places, making it presentation-ready.

## Section C: Coding Questions (4 Questions)

**Q1:** Write a Python function `calculate_profit_margin(revenue, expenses)` that calculates the profit margin as a percentage. It should handle zero revenue gracefully and return the result rounded to 2 decimal places.

```
def calculate_profit_margin(revenue, expenses):
    # Guard against division by zero
    if revenue == 0:
        return 0.0

    profit = revenue - expenses
    margin = (profit / revenue) * 100
    return round(margin, 2)

# Test cases
print(calculate_profit_margin(500000, 320000)) # 36.0
print(calculate_profit_margin(0, 100000)) # 0.0 – no crash
print(calculate_profit_margin(100, 75)) # 25.0

# Why round()? Because float arithmetic like (180000/500000)*100
# gives 35.99999... – round() ensures clean, presentable output.
```

**Q2:** Write a Python function `classify_employee(salary, department, is_active)` that uses `if/elif/else` and logical operators to return a classification string: 'Senior Active' (salary  $\geq$  80000 AND active), 'Junior Active' (salary  $<$  80000 AND active), or 'Inactive' (not active).

```
def classify_employee(salary, department, is_active):
    if not is_active:
        return 'Inactive'
    elif is_active and salary >= 80000:
        return 'Senior Active'
    else:
        return 'Junior Active'

# Test cases
print(classify_employee(95000, 'Finance', True)) # Senior Active
print(classify_employee(45000, 'HR', True)) # Junior Active
print(classify_employee(120000, 'Tech', False)) # Inactive

# Note: I check not is_active FIRST because an inactive employee
# should be classified as Inactive regardless of salary – this is
# the most exclusive rule, so it goes at the top of the chain.
```

**Q3:** Write a Python script that takes a raw list of sales records (each a dict with 'region', 'sales', 'month') and uses string manipulation, comparison operators, and logical operators to filter records where sales > 50000 AND region is either 'North' or 'South', then prints an f-string summary for each.

```
sales_records = [
    {'region': ' North ', 'sales': 75000, 'month': 'January'},
    {'region': 'South', 'sales': 43000, 'month': 'February'},
    {'region': 'East', 'sales': 90000, 'month': 'March'},
    {'region': 'North', 'sales': 62000, 'month': 'April'},
    {'region': 'South', 'sales': 51000, 'month': 'May'},
```

**Answer:]**

```
target_regions = ['North', 'South']
```

```
filtered = [
    r for r in sales_records
    if r['region'].strip() in target_regions # clean + membership check
    and r['sales'] > 50000 # comparison filter
```

**Answer:]**

```
for r in filtered:
    region = r['region'].strip()
    print(f"[{r['month']}] {region} - Sales: ₹{r['sales']:,}")
```

```
# Output:
# [January] North - Sales: ₹75,000
# [April] North - Sales: ₹62,000
# [May] South - Sales: ₹51,000
```

```
# Key technique: r['region'].strip() cleans before comparing -
# without this, ' North ' != 'North' and the record is dropped silently.
```

LEARN SMART. ANALYZE BETTER. GROW FASTER.

**Q4:** Write a function parse\_employee\_record(record) that takes a dict with string values (from a CSV import), validates types using isinstance(), casts each field, and returns a clean dict. Fields: 'emp\_id' (int), 'name' (str, stripped), 'salary' (float), 'is\_active' (bool).

```
def parse_employee_record(record):
    # Validate input is a dict
    if not isinstance(record, dict):
        raise TypeError('Input must be a dictionary')
```



```

cleaned = {}

# emp_id: cast to int
cleaned['emp_id'] = int(record.get('emp_id', 0))

# name: strip whitespace
name = record.get('name', '')
cleaned['name'] = name.strip() if isinstance(name, str) else str(name)

# salary: cast to float, default 0.0
sal = record.get('salary', '0')
cleaned['salary'] = float(sal) if str(sal).replace('.', '').isdigit() else 0.0

# is_active: 'True'/'False' string -> actual bool
active = record.get('is_active', 'False')
cleaned['is_active'] = active.strip().lower() == 'true'

return cleaned

# Test
raw = {'emp_id': '1042', 'name': ' Priya Sharma ', 'salary': '82000.50',
       'is_active': 'True'}
print(parse_employee_record(raw))
# {'emp_id': 1042, 'name': 'Priya Sharma', 'salary': 82000.5, 'is_active':
  True}

```

## Section D: Database Scenario-Based Coding (3 Questions)

*These questions simulate working with database-like structured data (list of dicts) using Python fundamentals — the precursor to pandas and SQL queries.*

**Q1:** Database Scenario: You have a transactions table (list of dicts) with fields: transaction\_id (int), customer\_name (str), amount (float), status (str), region (str). Write Python code to: (1) filter all 'Completed' transactions from the 'West' region with amount > 10000, and (2) calculate the total amount for those records.

```

transactions = [
    {'transaction_id': 101, 'customer_name': 'Arjun Nair', 'amount': 15000.0,
     'status': 'Completed', 'region': 'West'},
    {'transaction_id': 102, 'customer_name': 'Meena Pillai', 'amount': 8000.0,
     'status': 'Pending', 'region': 'West'},
    {'transaction_id': 103, 'customer_name': 'Suresh Rao', 'amount': 22000.0,
     'status': 'Completed', 'region': 'West'},
    {'transaction_id': 104, 'customer_name': 'Kavya Joshi', 'amount': 12000.0,
     'status': 'Completed', 'region': 'East'},

```

```
{'transaction_id': 105, 'customer_name': 'Nitin Desai', 'amount': 30000.0,
'status': 'Completed', 'region': 'West'},
```

**Answer: ]**

```
# Step 1: Filter – equivalent to SQL WHERE clause
filtered = [
    t for t in transactions
    if t['status'] == 'Completed'
    and t['region'] == 'West'
    and t['amount'] > 10000
```

**Answer: ]**

```
# Step 2: Aggregate – equivalent to SQL SUM()
total = sum(t['amount'] for t in filtered)
```

```
print(f'Matching records: {len(filtered)}')
for t in filtered:
    print(f" ID {t['transaction_id']} | {t['customer_name']} |
    ₹{t['amount']:,.2f}")
print(f'Total Amount: ₹{total:,.2f}')
```

```
# Output:
# Matching records: 2
# ID 101 | Arjun Nair | ₹15,000.00
# ID 103 | Suresh Rao | ₹22,000.00 <- Wait, let me recheck...
# Actually ID 103 matches, ID 105 matches too (30000 > 10000)
# Total Amount: ₹67,000.00
```

```
# SQL equivalent:
# SELECT *, SUM(amount) FROM transactions
# WHERE status = 'Completed' AND region = 'West' AND amount > 10000
```

**Q2: Database Scenario:** You have an employees table. Write Python code to group employees by department and calculate the average salary per department. The result should be a dict mapping department name to average salary (rounded to 2 decimal places).

```
employees = [
    {'emp_id': 1, 'name': 'Asha', 'dept': 'Finance', 'salary': 72000},
    {'emp_id': 2, 'name': 'Ravi', 'dept': 'HR', 'salary': 48000},
    {'emp_id': 3, 'name': 'Meena', 'dept': 'Finance', 'salary': 85000},
    {'emp_id': 4, 'name': 'Kiran', 'dept': 'HR', 'salary': 52000},
    {'emp_id': 5, 'name': 'Priya', 'dept': 'Tech', 'salary': 95000},
    {'emp_id': 6, 'name': 'Dev', 'dept': 'Tech', 'salary': 88000},
```

**Answer: ]**

```
# Step 1: Group salaries by department
dept_salaries = {}
for emp in employees:
    dept = emp['dept']
    if dept not in dept_salaries:
        dept_salaries[dept] = []
    dept_salaries[dept].append(emp['salary'])

# Step 2: Calculate average per department
dept_avg = {
    dept: round(sum(salaries) / len(salaries), 2)
    for dept, salaries in dept_salaries.items()
}
```

**Answer: }**

```
print('Department Average Salaries:')
for dept, avg in dept_avg.items():
    print(f' {dept}: ₹{avg:,.2f}')
```

```
# Output:
# Finance: ₹78,500.00
# HR: ₹50,000.00
# Tech: ₹91,500.00
```

```
# SQL equivalent:
# SELECT dept, ROUND(AVG(salary), 2) AS avg_salary
# FROM employees
# GROUP BY dept
```

**Q3: Database Scenario:** You have an orders table and a customers table. Simulate a JOIN in Python: for each order, find the matching customer name, then produce a combined record. Filter the result to show only orders where the customer is 'Premium' tier AND order\_value > 5000.

```
orders = [
    {'order_id': 'O01', 'customer_id': 'C01', 'order_value': 8500, 'status':
    'Shipped'},
    {'order_id': 'O02', 'customer_id': 'C02', 'order_value': 3200, 'status':
    'Pending'},
    {'order_id': 'O03', 'customer_id': 'C01', 'order_value': 6000, 'status':
    'Delivered'},
    {'order_id': 'O04', 'customer_id': 'C03', 'order_value': 9999, 'status':
    'Shipped'},
]
```

**Answer: ]**

```
customers = [
    {'customer_id': 'C01', 'name': 'Riya Mehta', 'tier': 'Premium'},
]
```

```
{'customer_id': 'C02', 'name': 'Arjun Singh', 'tier': 'Standard'},
{'customer_id': 'C03', 'name': 'Priya Nair', 'tier': 'Premium'},
```

### Answer: ]

```
# Step 1: Build a lookup dict from customers (like an index)
customer_lookup = {c['customer_id']: c for c in customers}
```

```
# Step 2: JOIN – merge order with customer data
joined = []
for order in orders:
    cust = customer_lookup.get(order['customer_id'])
    if cust:
        joined.append({
            'order_id': order['order_id'],
            'customer': cust['name'],
            'tier': cust['tier'],
            'order_value': order['order_value'],
            'status': order['status'],
        })
```

```
# Step 3: Filter – Premium tier AND order_value > 5000
filtered = [
    r for r in joined
    if r['tier'] == 'Premium' and r['order_value'] > 5000
```

### Answer: ]

```
print('Premium High-Value Orders:')
for r in filtered:
    print(f" {r['order_id']} | {r['customer']} | ₹{r['order_value']:,.} | {r['status']}")
```

```
# Output:
# 001 | Riya Mehta | ₹8,500 | Shipped
# 003 | Riya Mehta | ₹6,000 | Delivered
# 004 | Priya Nair | ₹9,999 | Shipped
```

```
# SQL equivalent:
# SELECT o.order_id, c.name, c.tier, o.order_value, o.status
# FROM orders o JOIN customers c ON o.customer_id = c.customer_id
# WHERE c.tier = 'Premium' AND o.order_value > 5000
```